Informe de optimización y preparación para producción de PharmaSoft

Lenguaje de Programación II · Sesión 13 · Actividad autónoma, Reto 02
Estudiante: Omar Tineo · Noviembre de 2026

Resumen para la gerencia

- **Qué se ganó:** recargar `/productos` transfiere 2.6 kB en lugar de 160 kB, y el 95 % de las lecturas repetidas de catálogo ya no consulta Oracle.
- **Qué no mejoró:** el tiempo de respuesta percibido en esta instalación es casi el mismo, porque Oracle está en la misma máquina y cada petición sigue haciendo 3 consultas para validar al usuario.
- **Datos correctos:** los 6 casos de la matriz de invalidación y el caso extra de escritura con Redis caído muestran el dato correcto.
- **Defecto corregido:** el log de producción escribía el DNI completo al registrar un cliente.
- **Antes de instalar en los tres locales:** faltan HTTPS, contraseña en Redis, copias de seguridad de Oracle y alertas.

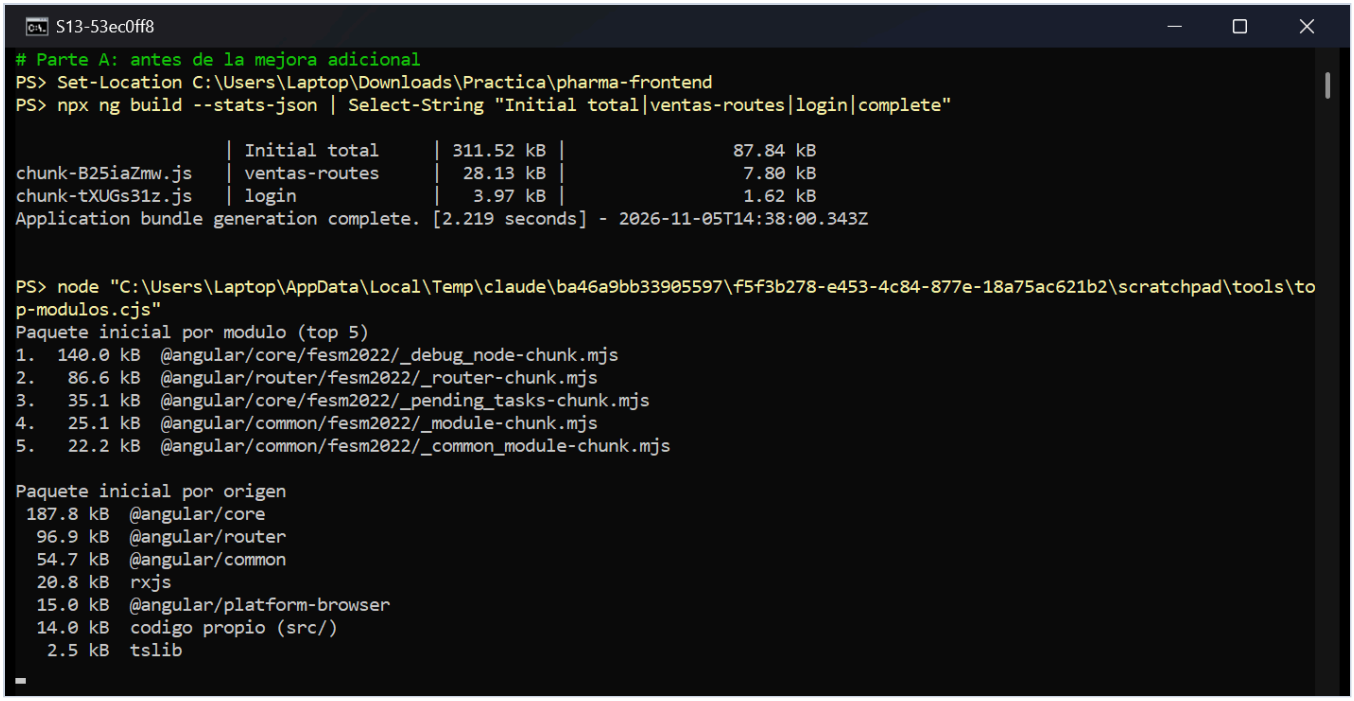
Parte A. Carga diferida y tamaño de entrega

A.1 Cinco módulos más pesados del paquete inicial

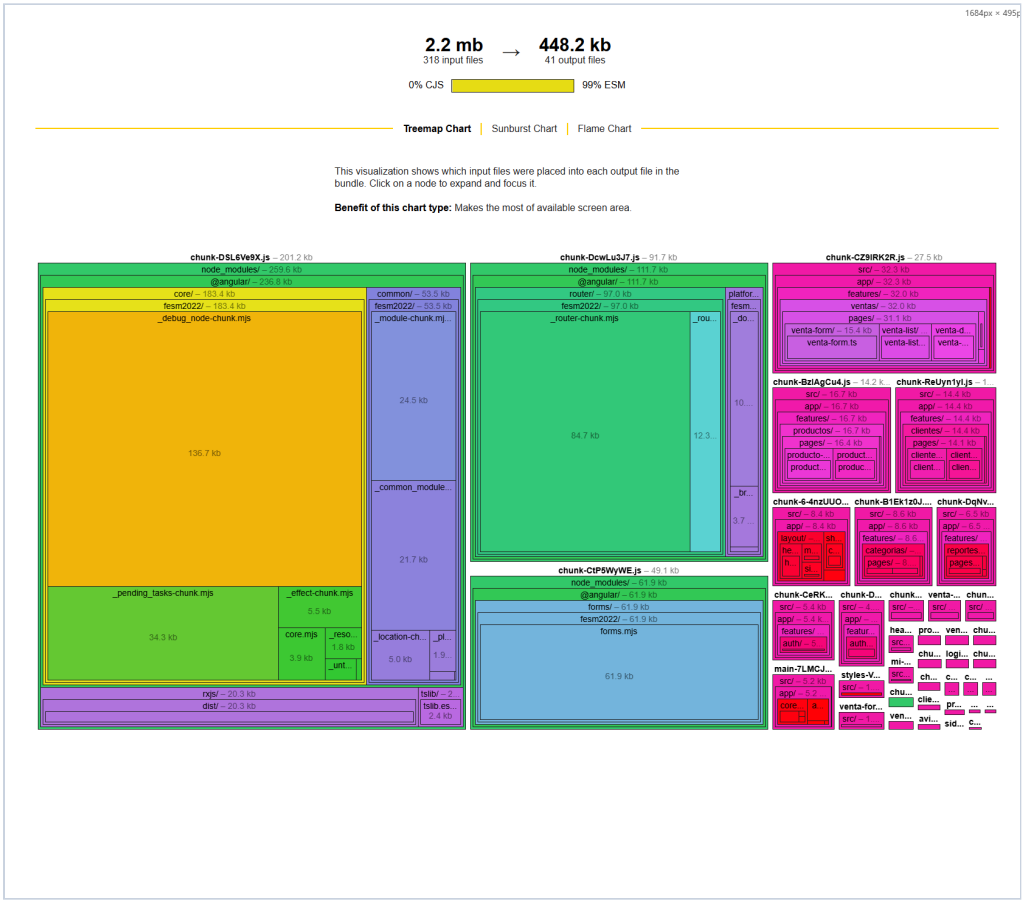
Datos de `browser-stats.json`, generado con `ng build --stats-json`. Se ordenaron con un script por bytes en la salida y se visualizaron en el analizador de esbuild.

N.º	Módulo	Tamaño	Depende de	Para qué sirve
1	`@angular/core/fesm2022/_debug_node-chunk.mjs`	140.0 kB	Angular	Motor de renderizado y detección de cambios
2	`@angular/router/fesm2022/_router-chunk.mjs`	86.6 kB	Angular	Enrutador
3	`@angular/core/fesm2022/_pending_tasks-chunk.mjs`	35.1 kB	Angular	Inyección de dependencias y tareas internas
4	`@angular/common/fesm2022/_module-chunk.mjs`	25.1 kB	Angular	HttpClient
5	`@angular/common/fesm2022/_common_module-chunk.mjs`	22.2 kB	Angular	Directivas y pipes comunes

Los cinco son de Angular. Por origen: `@angular/core` 187.8 kB, `@angular/router` 96.9 kB, `@angular/common` 54.7 kB, RxJS 20.8 kB, `@angular/platform-browser` 15.0 kB, código propio 14.0 kB y tslib 2.5 kB. No hay librerías de terceros pesadas que reemplazar, y lo único reducible es el código propio.



A-01_antes_stats-y-modulos.png · Antes de la mejora: Initial total y módulos.



A-analyzer.png · esbuild Bundle Analyzer con browser-stats.json.

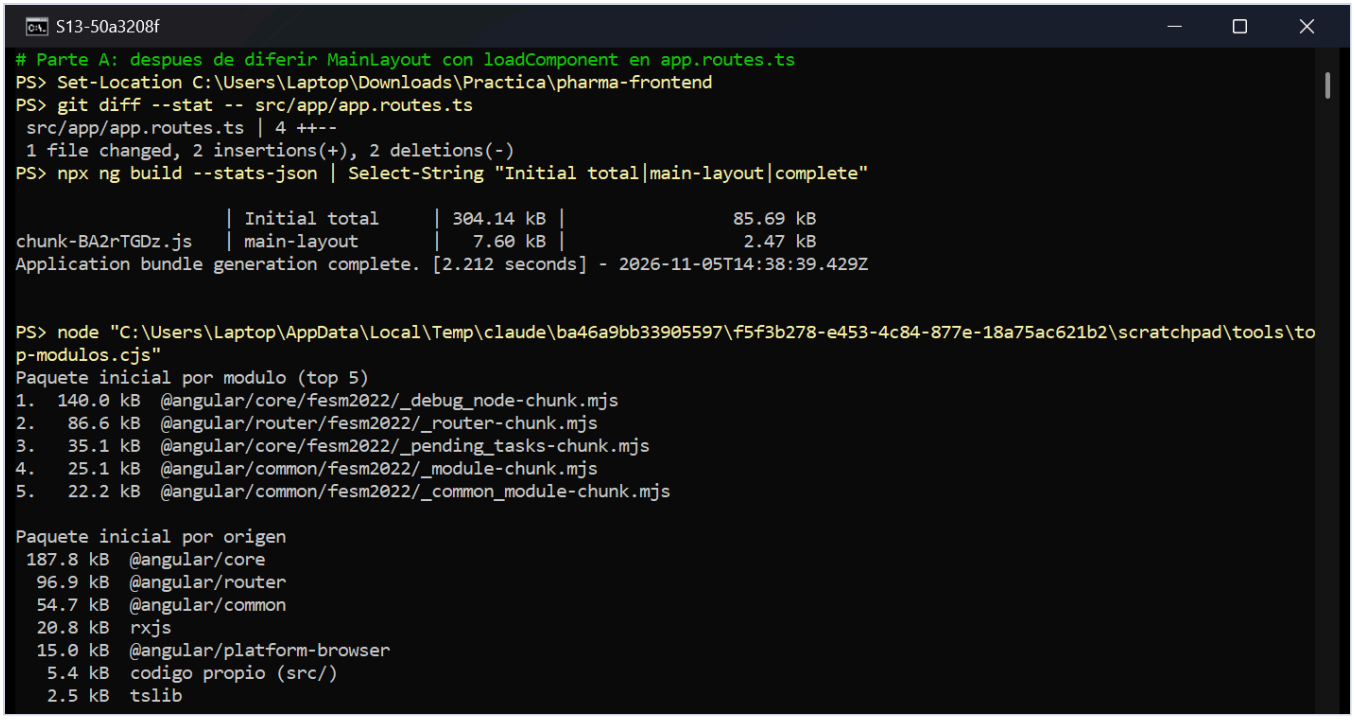
A.2 Mejora adicional: diferir MainLayout

`app.routes.ts` importaba `MainLayout` de forma estática. Por eso la cabecera, el menú y el aviso de sesión viajaban en el paquete inicial, aunque el login no los usa. Se cambió `component` por `loadComponent`.

Medición (ng build)	Antes	Después	Diferencia
Initial total, Raw	311.52 kB	304.14 kB	−7.38 kB (−2.4 %)
Initial total, Transfer	87.84 kB	85.69 kB	−2.15 kB (−2.4 %)

Código propio en el paquete inicial	14.0 kB	5.4 kB	−8.6 kB
Nuevo chunk diferido `main-layout`	—	7.60 kB	

La mejora es real pero modesta, porque el resto del paquete es Angular. Con la precarga elegida en A.3, el Initial total final queda en **306.15 kB / 86.07 kB**.



A-03_despues_stats-y-modulos.png · Después de diferir MainLayout.

A.3 Navegación con y sin precarga

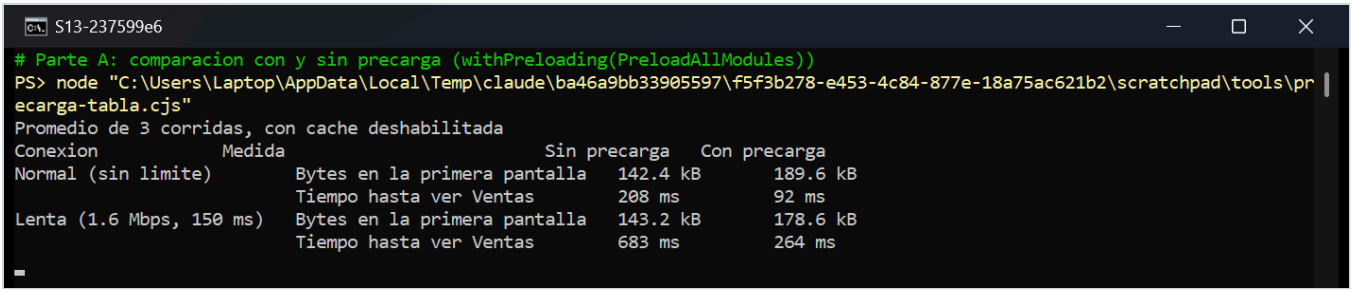
Promedio de 3 corridas con caché deshabilitada. «Primera pantalla» suma todos los bytes desde `/login` hasta el inicio con la red en reposo. «Hasta ver Ventas» se mide desde el clic en el menú, 2 s después de iniciar sesión, hasta que aparece la tabla.

Conexión	Medida	Sin precarga	Con precarga
Normal	Bytes de la primera pantalla	142.4 kB	189.6 kB
Normal	Tiempo hasta ver Ventas	208 ms	92 ms
Lenta (1.6 Mbps, 150 ms)	Bytes de la primera pantalla	143.2 kB	178.6 kB
Lenta (1.6 Mbps, 150 ms)	Tiempo hasta ver Ventas	683 ms	264 ms

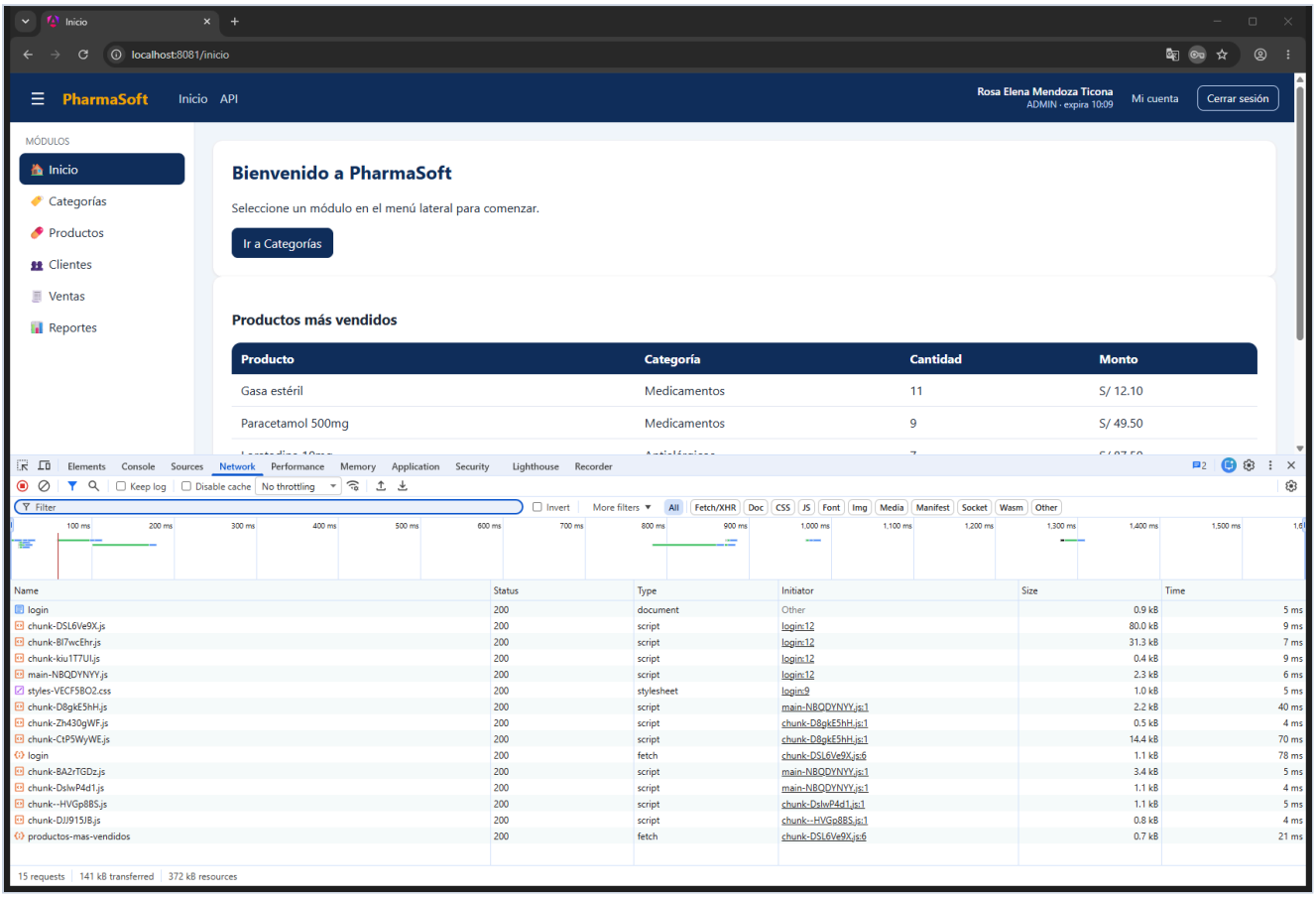
Decisión: con precarga. En el local de la periferia la espera hasta Ventas baja un 61 % (de 683 a 264 ms) a cambio de unos 36 a 47 kB más, que se descargan en segundo plano después de mostrar la primera pantalla. Los JS quedan un año en caché, así que ese costo se paga una vez por cada despliegue, no en cada visita. Las computadoras de la botica usan siempre los mismos módulos.

Costo aceptado: `PreloadAllModules` también descarga módulos que el rol no usa, por ejemplo Reportes para el vendedor. Es código, no datos: el acceso sigue protegido por el guard y el backend. El bloque `@defer` no se precarga: se comprobó que el vendedor sigue sin descargar `mas-vendidos` ni llamar al reporte (A-06).

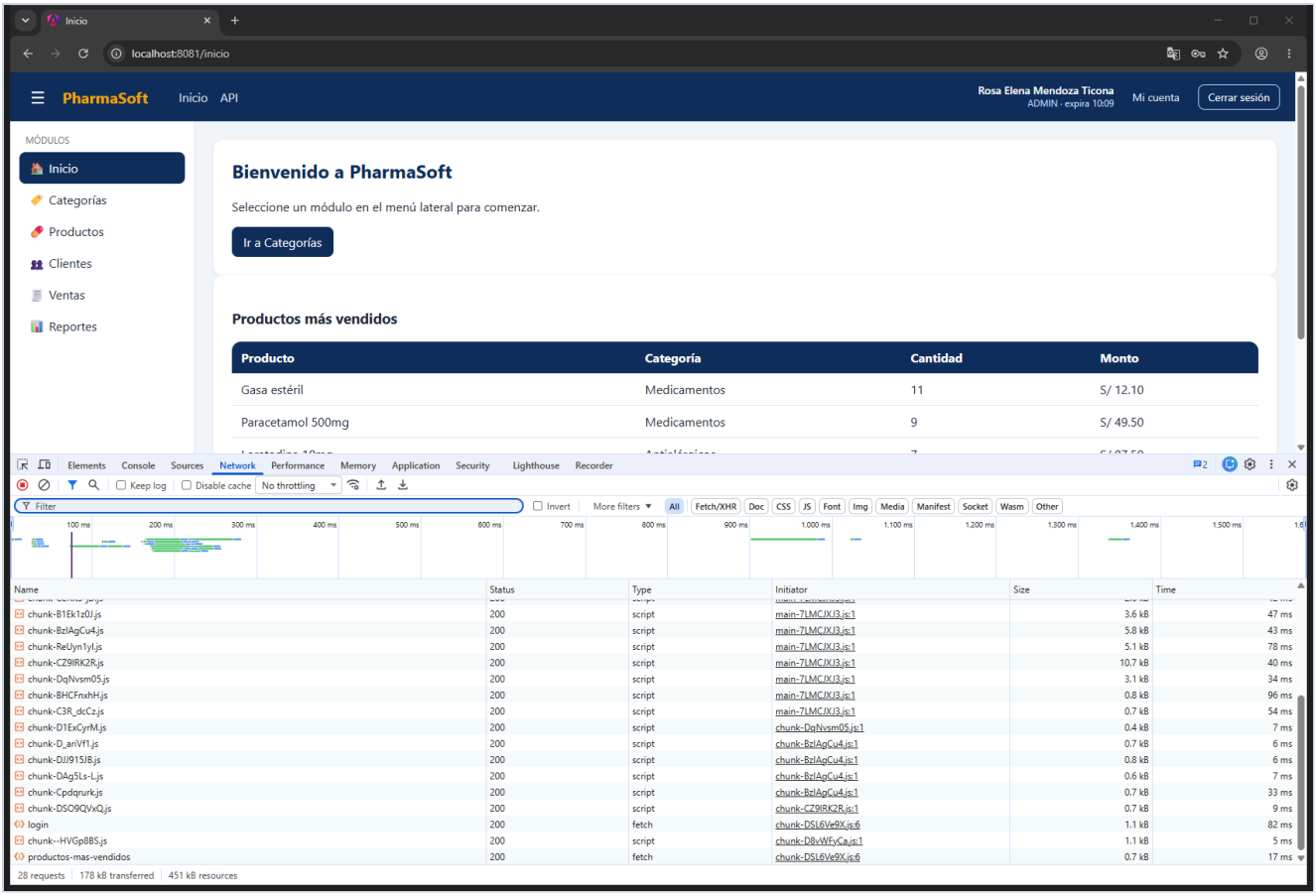
> Con la conexión lenta, la cifra de 178.6 kB puede no incluir todos los módulos, porque la espera de 2 s pudo terminar antes de completarse la precarga. Por eso el costo se indica como un rango.



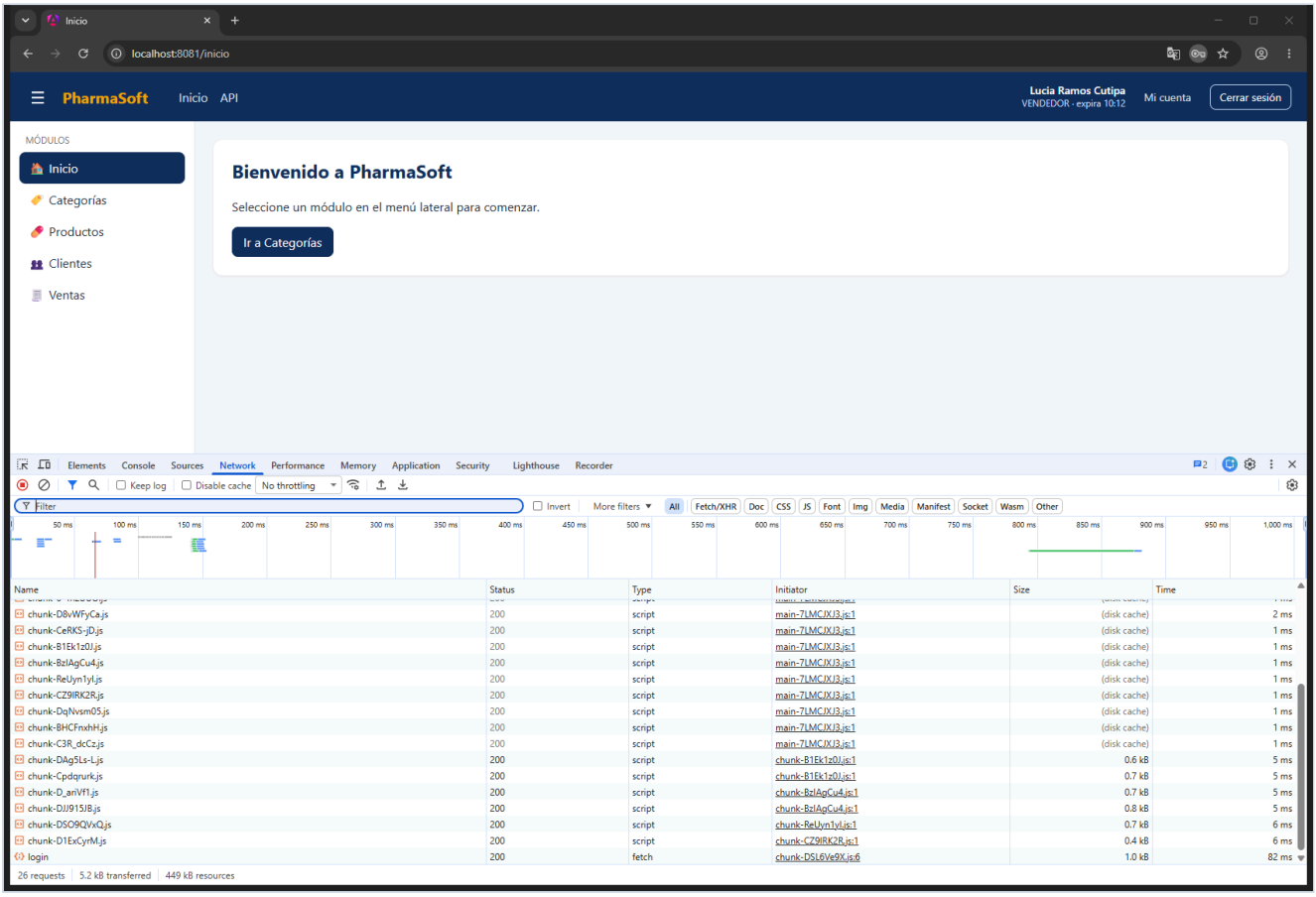
A-04_precarga_cifras.png · Tabla de resultados.



A-05_precarga-sin_primera-pantalla.png · Sin precarga: red de la primera pantalla.



A-05_precarga-con_primera-pantalla.png · Con precarga: red de la primera pantalla.

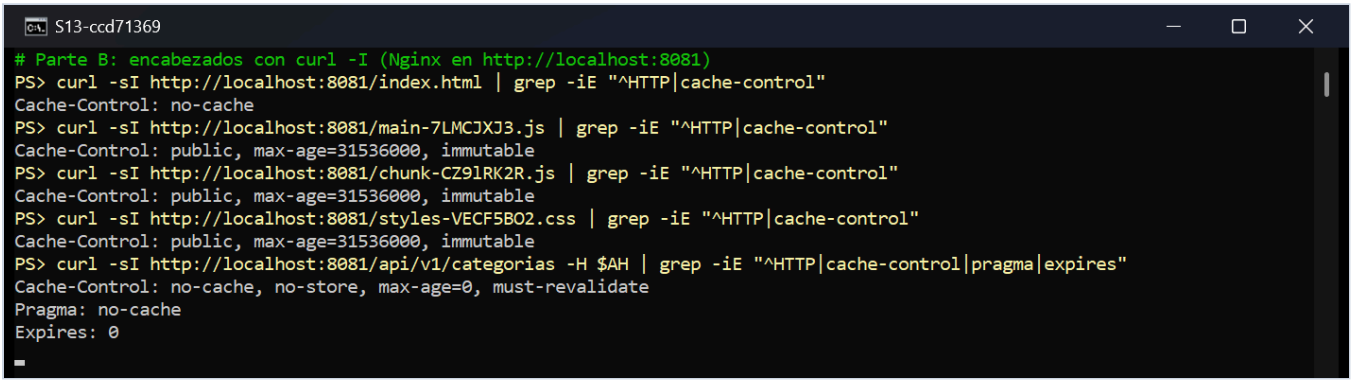


A-06_con-precarga_vendedor-red.png · Con precarga, vendedor: 0 descargas de mas-vendidos y 0 llamadas al reporte.

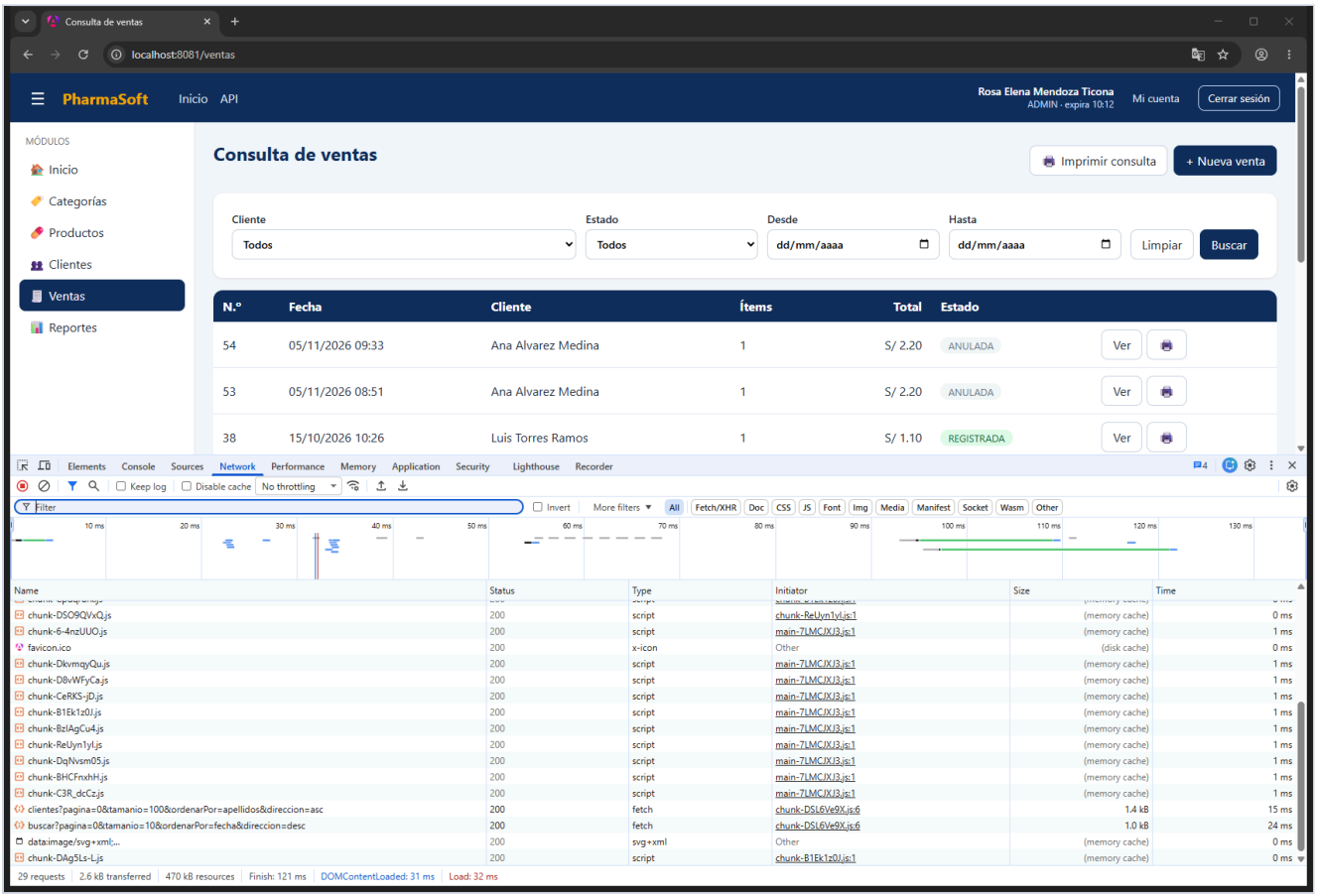
Parte B. Políticas de caché del navegador

Recurso	Cache-Control obtenido	Origen en la recarga	¿Correcto?
/index.html	`no-cache`	304 (revalidado con ETag)	Correcto
main-*.js	`public, max-age=31536000, immutable`	Memoria (*memory cache*)	Correcto
chunk-* de ventas	`public, max-age=31536000, immutable`	Memoria (*memory cache*)	Correcto
styles-*.css	`public, max-age=31536000, immutable`	Memoria (*memory cache*)	Correcto
/api/v1/categorias	`no-cache, no-store, max-age=0, must-revalidate`	Red (200)	Correcto

El documento `/productos` es `index.html` servido por `try_files`, y por eso responde 304. El favicon sale de *disk cache*. Con F5 sobre la misma pestaña Chrome usa la caché de memoria; esta prueba no midió la caché de disco para los JS.



B-01_curl-encabezados.png · curl -I de los cinco recursos.



B-02_recarga-ventas_origen.png · Recarga en /ventas: origen de cada recurso.



B-03_recarga-productos_api.png · Recarga en /productos: las llamadas al API van a la red.

Despliegue simulado

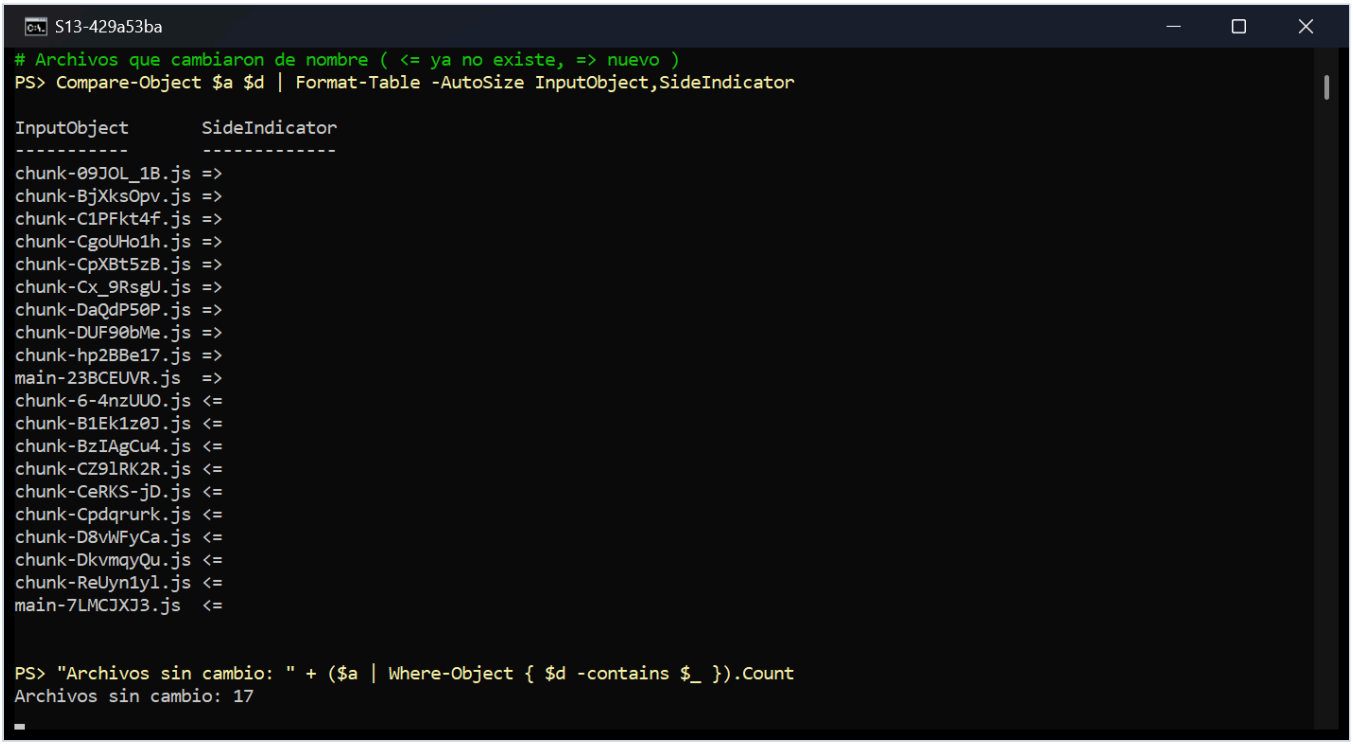
Se cambió el título de la página de Productos a «Productos de la botica» y se ejecutó `ng build` con Nginx en marcha. Después se revirtió el cambio.

Resultado	Valor
Archivos que cambiaron de nombre	10 (`main-*` y 9 `chunk-*`)
Archivos sin cambio	17
Transferido al recargar	40.8 kB (28 peticiones)

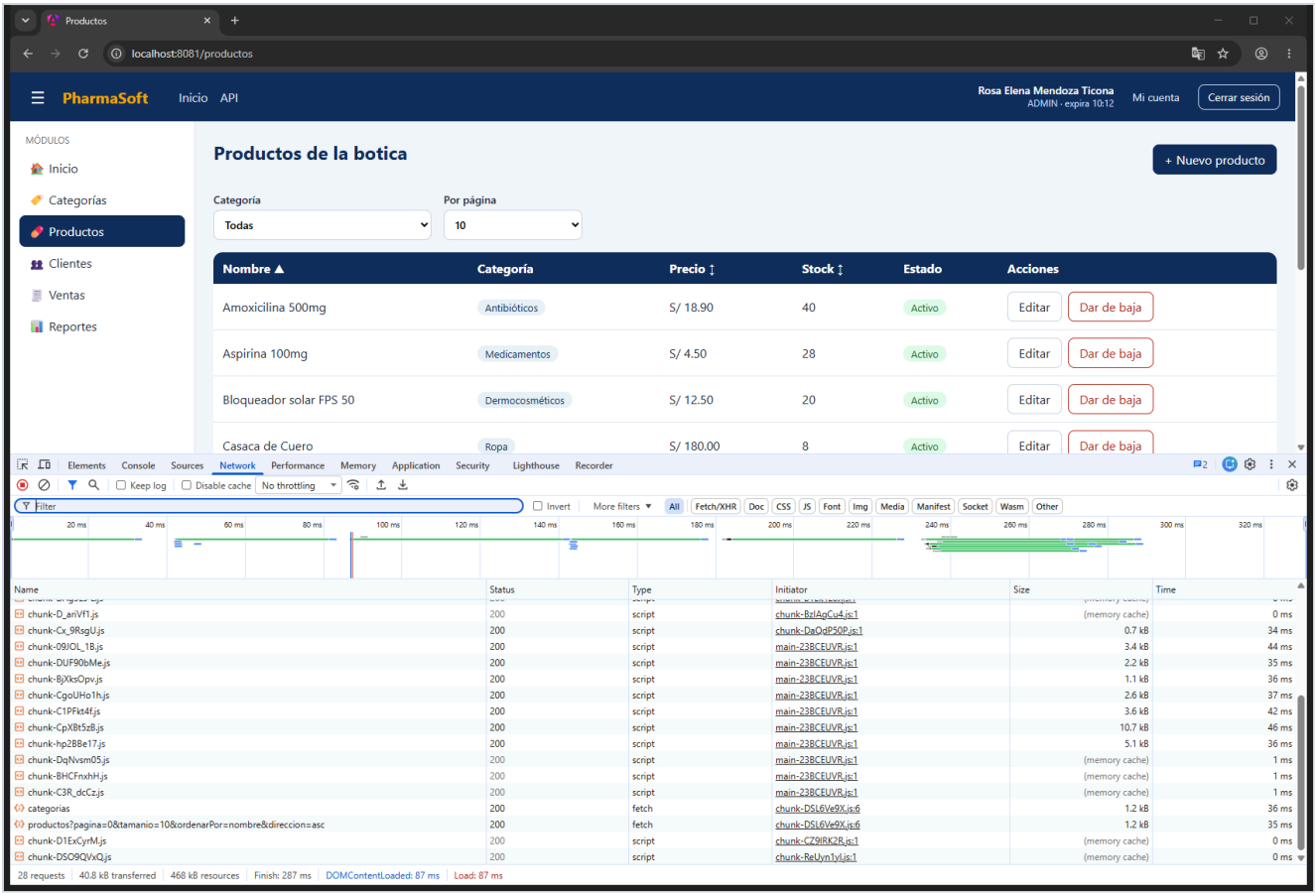
- **Por qué cambiaron 10 archivos por un texto:** cada archivo incluye los nombres con hash de los que importa. Al cambiar el chunk de Productos cambia su hash, luego el de quien lo importa, y así hasta `main-*`.
- **Qué se descargó:** solo los archivos con nombre nuevo (200 con tamaño en B-06). Los que no cambiaron siguen saliendo de *memory cache*.
- **Por qué se ve la versión nueva sin borrar la caché:** `index.html` se revalida siempre (`no-cache`), recibe la lista nueva de archivos y el navegador pide esos nombres, que no tiene guardados. `immutable` es seguro porque un archivo con un nombre dado nunca cambia de contenido.



B-04_antes-del-despliegue.png · Antes del despliegue: «Productos».



B-05_despliegue_archivos.png · Archivos nuevos (=>) y eliminados (<=), y archivos sin cambio.



B-06_recarga-tras-despliegue.png · Recarga: se ve el título nuevo; descargas con tamaño y resto desde memoria.

Por qué el API no se guarda en la caché del navegador

- **Datos que cambian:** un stock guardado llevaría a vender lo que ya no hay. Con `no-store` cada lectura va al backend, que responde desde Redis u Oracle y es quien invalida.
- **Datos privados:** las respuestas dependen de quién inició sesión. En una computadora compartida de la botica, una copia en disco quedaría disponible para el siguiente usuario.

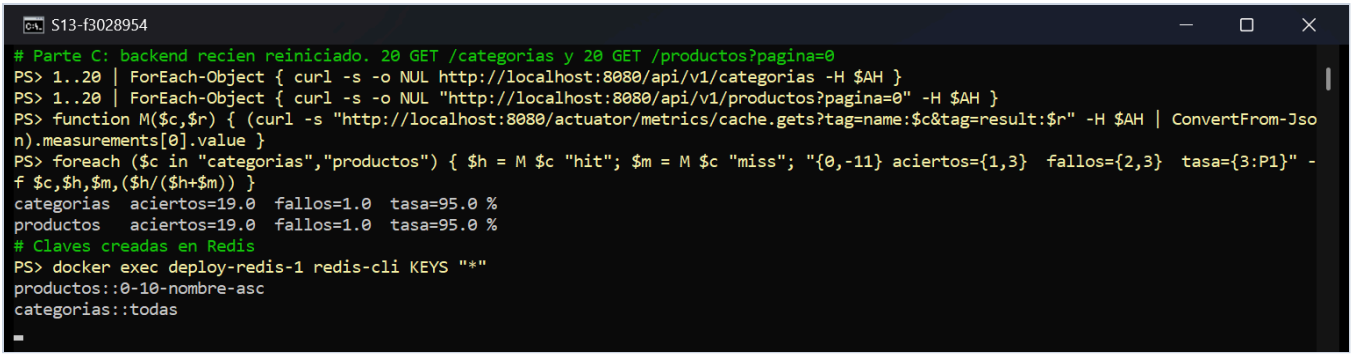
Spring Security agrega por defecto `Cache-Control: no-cache, no-store, max-age=0, must-revalidate`, `Pragma: no-cache` y `Expires: 0`, como se ve en B-01.

Parte C. Caché con Redis

Aciertos y fallos

Caché	Aciertos	Fallos	Tasa de aciertos
categorias	19	1	95.0 %
productos	19	1	95.0 %

Con 20 peticiones iguales, el único fallo es la primera, que llena la caché; las otras 19 aciertan hasta que expire el TTL o una escritura invalide. La tasa real en la botica será menor, porque cada venta limpia la caché de productos y hay varias combinaciones de página y orden.

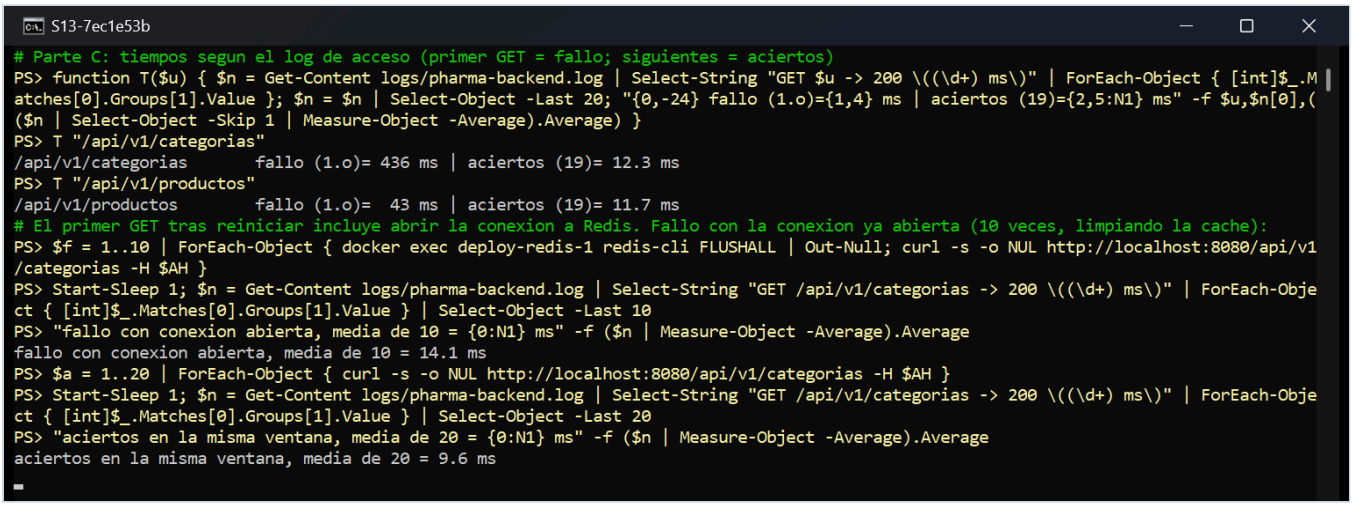


C-aciertos.png · 20 GET por caché y métrica cache.gets filtrada por name y result.

Tiempo de respuesta (log de acceso)

Caso	Tiempo medio
/categorias, primer GET tras reiniciar (fallo)	436 ms
/categorias, siguientes 19 (aciertos)	12.3 ms
/productos, primer GET (fallo)	43 ms
/productos, siguientes 19 (aciertos)	11.7 ms
/categorias, fallo con la conexión ya abierta (10 veces)	14.1 ms
/categorias, aciertos en la misma ventana (20 veces)	9.6 ms

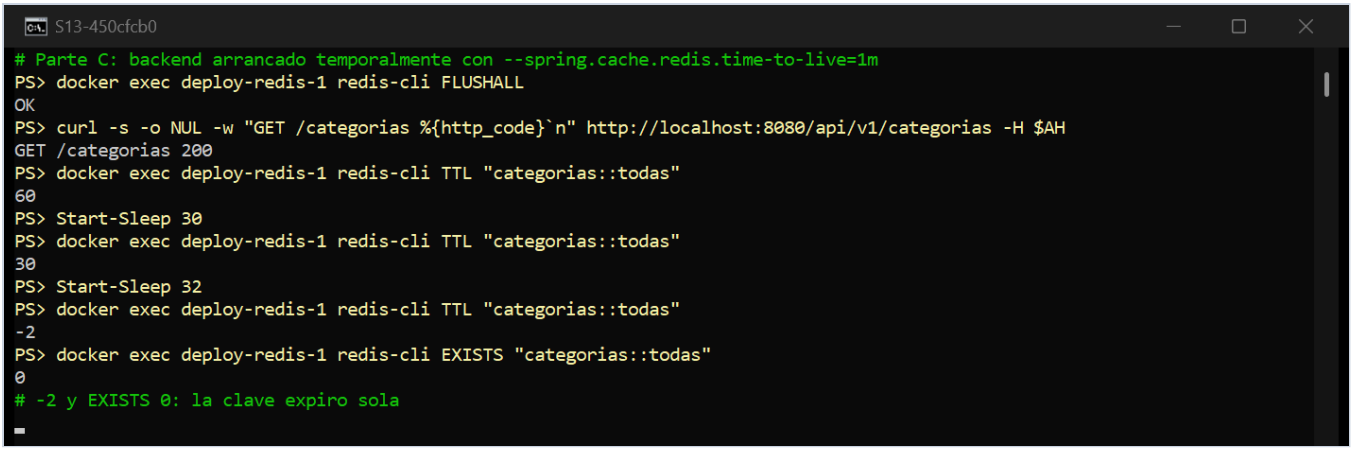
Los 436 ms no se deben a la consulta: incluyen abrir la conexión a Redis y cargar código por primera vez. La comparación justa es 14.1 ms contra 9.6 ms, un 32 % menos. En esta máquina Oracle es local y la tabla tiene 12 filas, así que la ganancia absoluta es pequeña. La ganancia principal es la carga que se le quita a Oracle en horas punta.



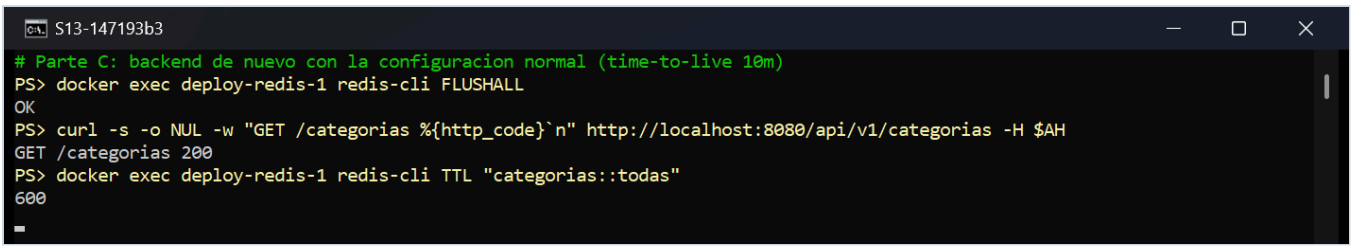
C-tiempos.png · Tiempos calculados desde el log de acceso.

Vigencia (TTL)

Se arrancó el backend con `--spring.cache.redis.time-to-live=1m`, sin cambiar los archivos del repositorio, y luego se volvió a 10 minutos.



C-ttl-1m.png · TTL de 1 minuto: la clave expira sola (TTL -2, EXISTS 0).



C-ttl-10m.png · De nuevo con 10 minutos: TTL cercano a 600 s.

Matriz de invalidación

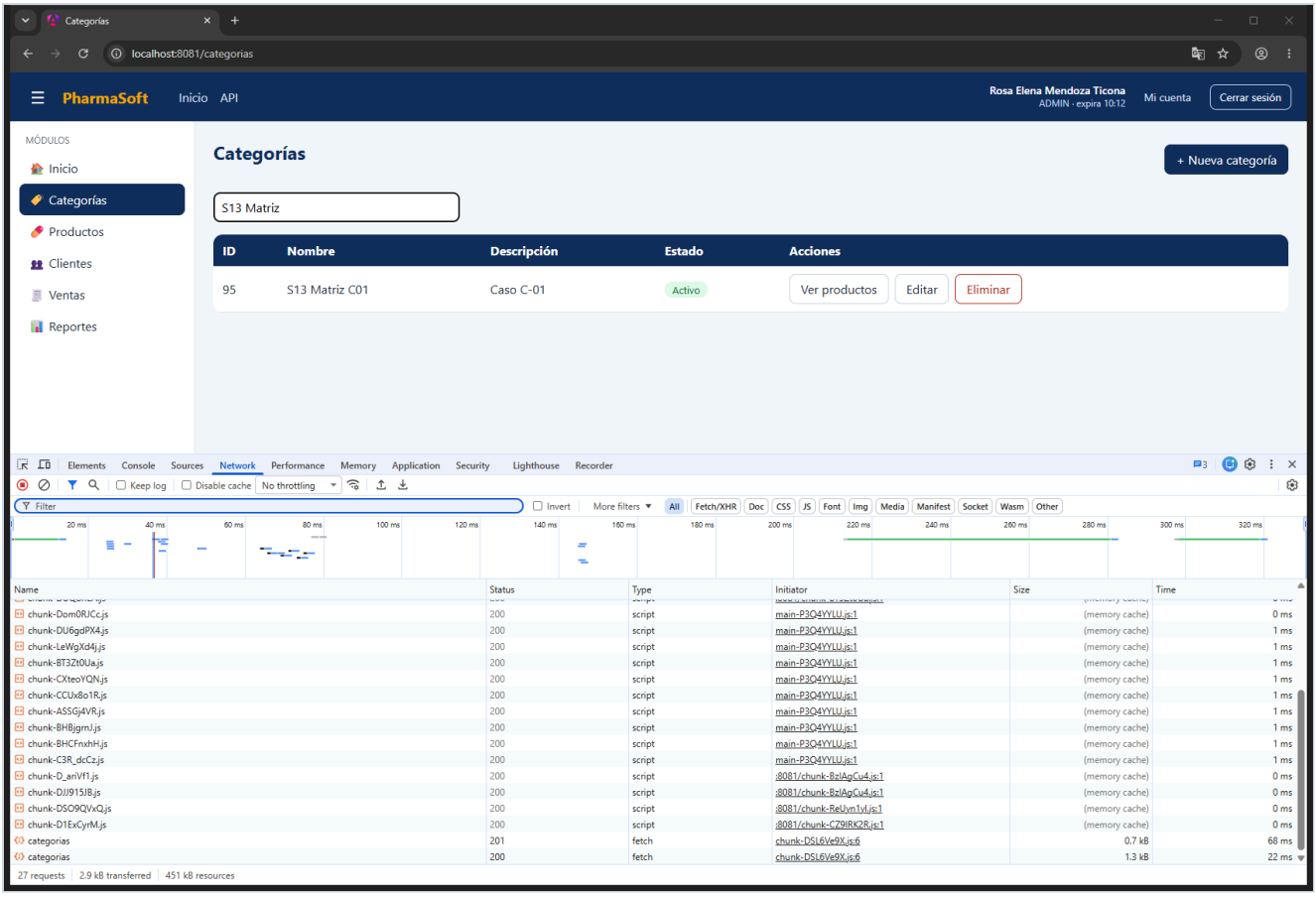
Antes de cada operación se visitó la lista para que la caché estuviera poblada; de lo contrario la prueba no demostraría nada.

ID	Operación	Se verifica	Esperado	Obtenido	Captura
C-01	Crear una categoría	Lista de categorías	Aparece	Aparece «S13 Matriz C01» Correcto	C-01
C-02	Cambiar el nombre de una categoría	Columna Categoría de productos	Nombre nuevo	Loratadina: «Antialérgicos» → «Antialérgicos S13» Correcto	C-02a, C-02b
C-03	Editar el precio de un producto	GET /productos/{id} y la lista	Precio nuevo	Aspirina: S/ 4.50 → S/ 4.75 en ambos Correcto	C-03a, C-03b
C-04	Registrar una venta	Stock en la lista	Descontado	Gasa estéril: 90 → 87 Correcto	C-04
C-05	Eliminar una categoría sin productos	Lista de categorías	Ya no aparece	0 filas; ausente en el API Correcto	C-05
C-06	Detener Redis y listar productos	SPA y /actuator/health	Funciona; DOWN	10 filas en la SPA; health 503 DOWN Correcto	C-06a, C-06b
Extra	Anular una venta	Stock en la lista	Repuesto	Gasa estéril: 87 → 90 Correcto	C-04b
C-07 (extra)	Editar un precio con Redis caído y volver a iniciarlo	Lista tras reconectar	Precio nuevo	S/ 4.60 Correcto	C-07

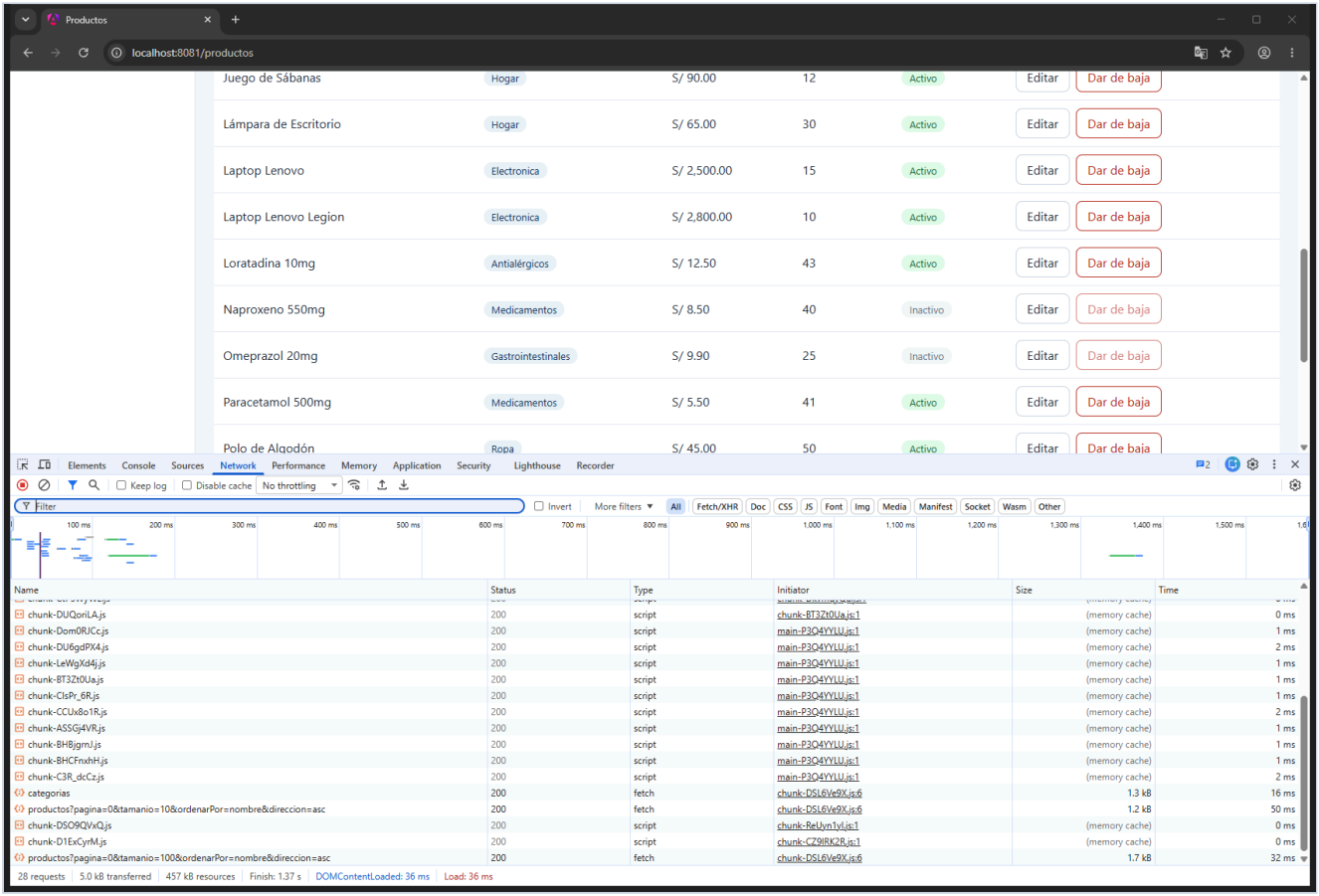
Defectos: ninguno en las anotaciones `@CacheEvict`. Se revisaron también las escrituras que la guía no enumera y por eso, ya desde el Reto 01, `anular` invalida productos porque repone stock (la guía solo pedía `registrar`).

C-07: se esperaba encontrar un precio antiguo, pero no ocurrió. Al volver Redis, la clave de productos ya no estaba, mientras que `categorias::todas` sobrevivió al reinicio. La explicación más probable es que el cliente

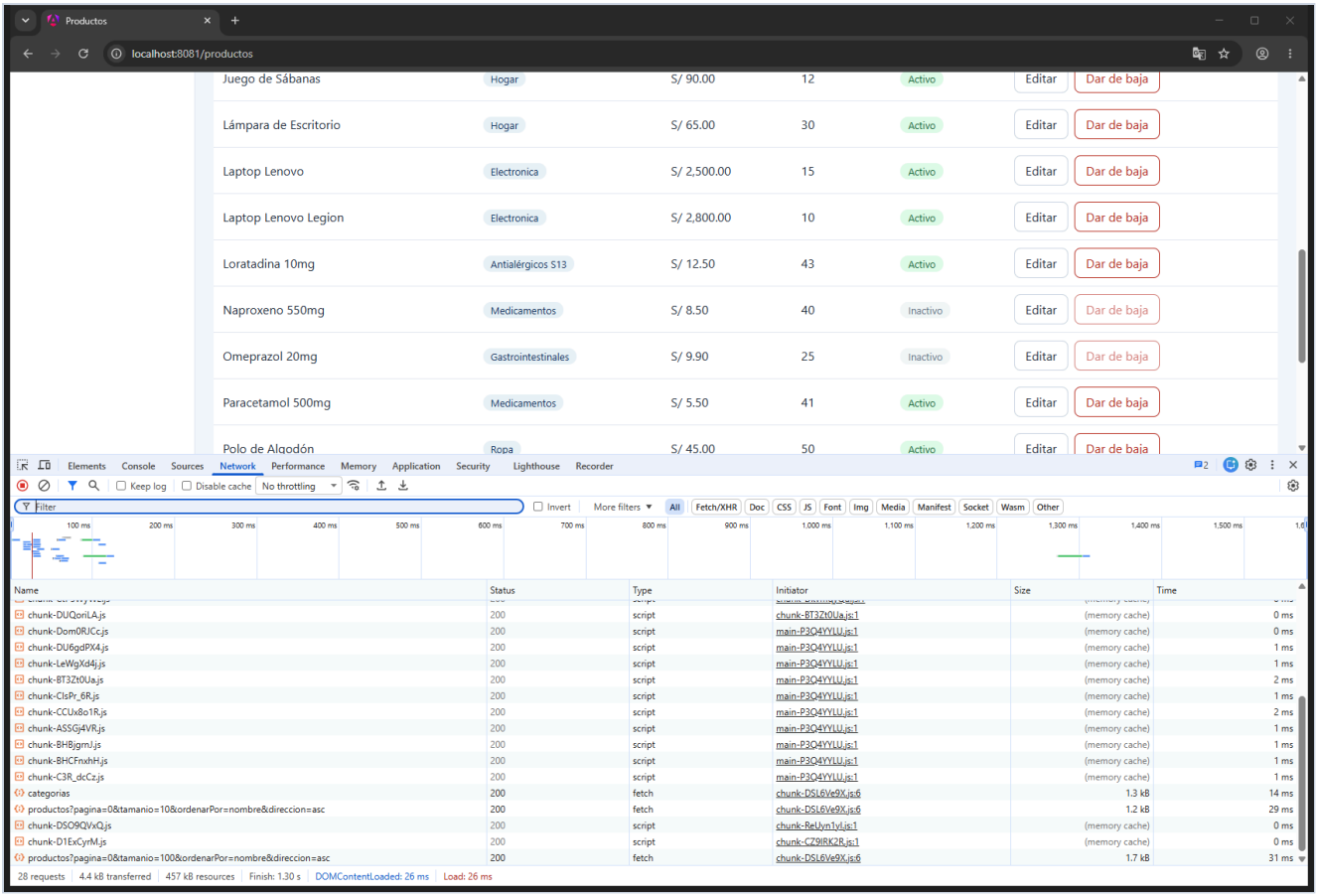
Lettuce mantiene la orden de limpieza en cola y la ejecuta al reconectar; esto se dedujo de lo observado, no del código del cliente. **No se probó** reiniciar el backend mientras Redis está caído: en ese caso la cola se perdería y el dato antiguo podría durar hasta 10 minutos, que es justo para lo que sirve el TTL.



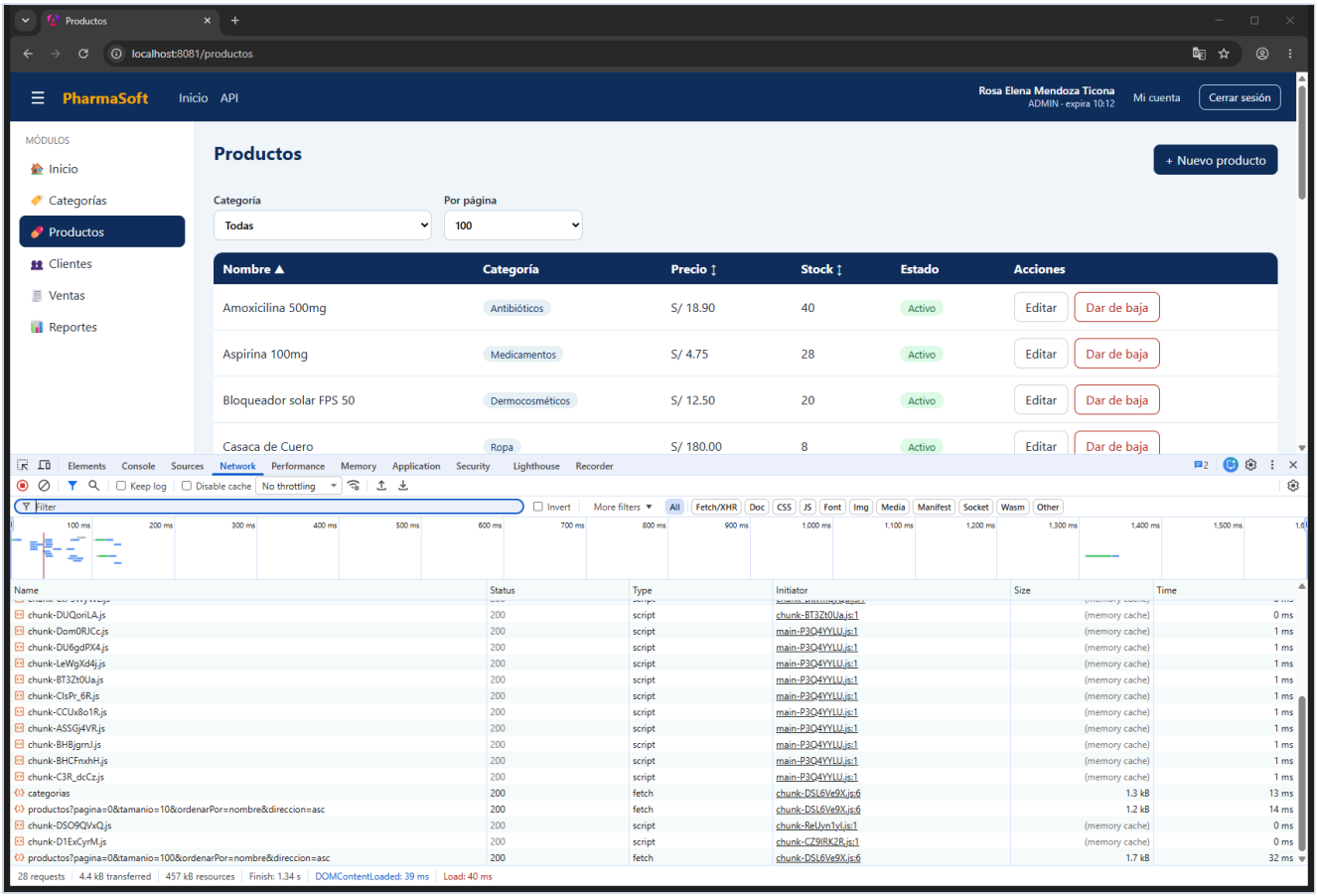
C-01-crear-categoria.png · C-01.



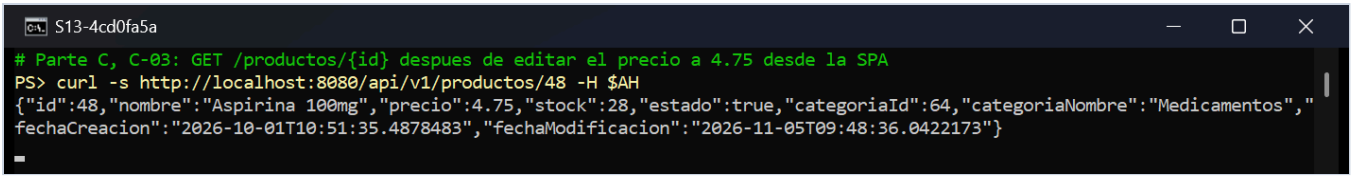
C-02a-antes.png · C-02, antes.



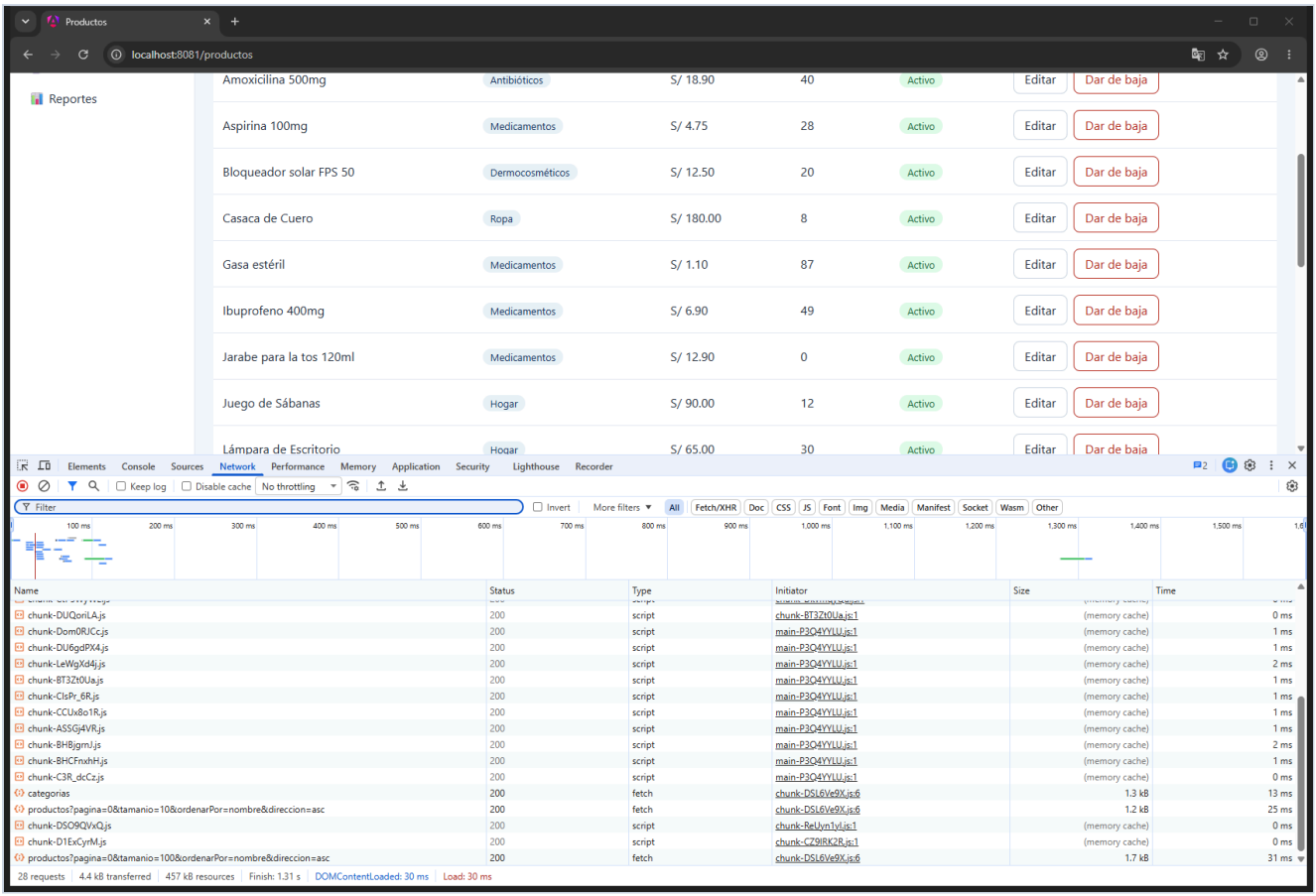
C-02b-nombre-nuevo.png · C-02, nombre nuevo.



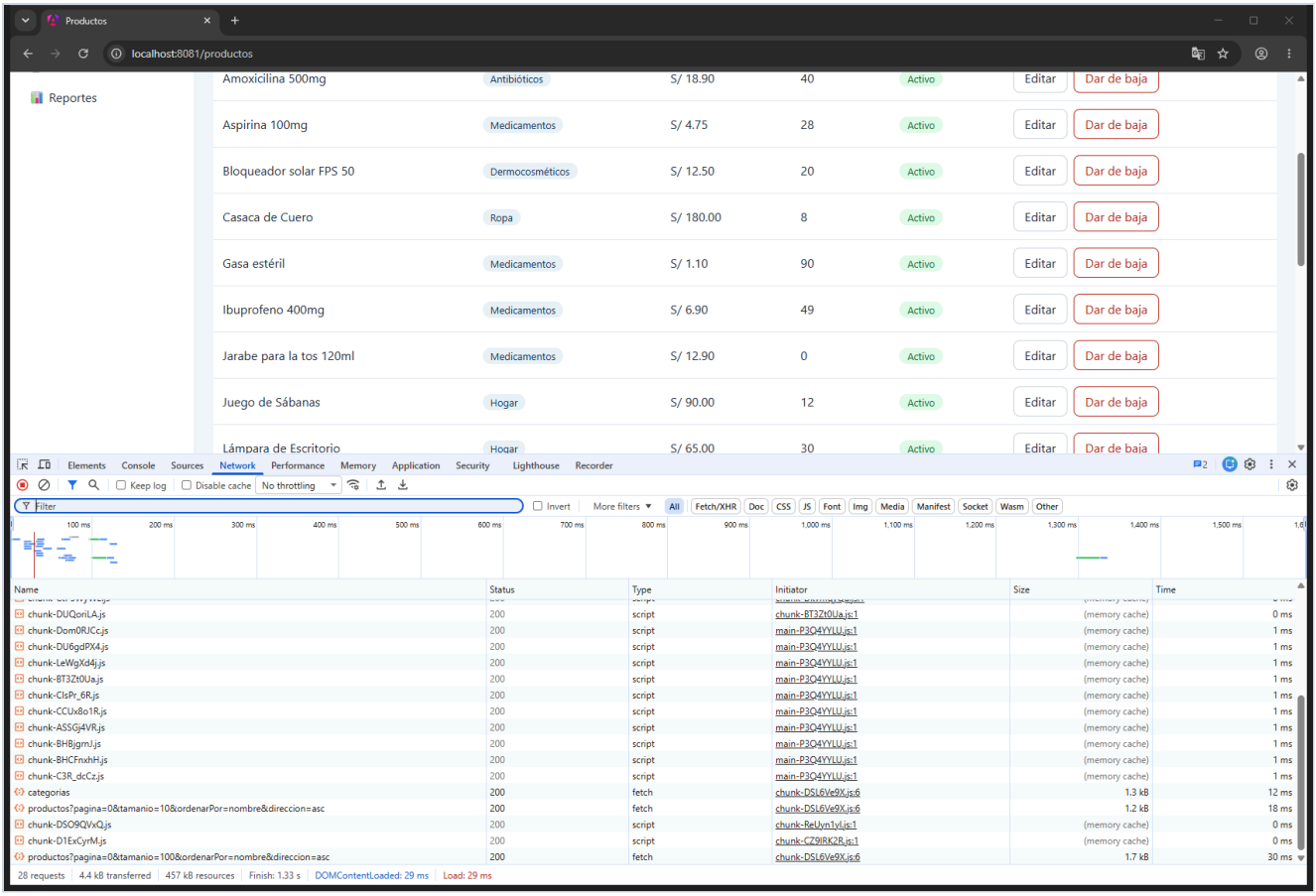
C-03a-precio-lista.png · C-03, lista.



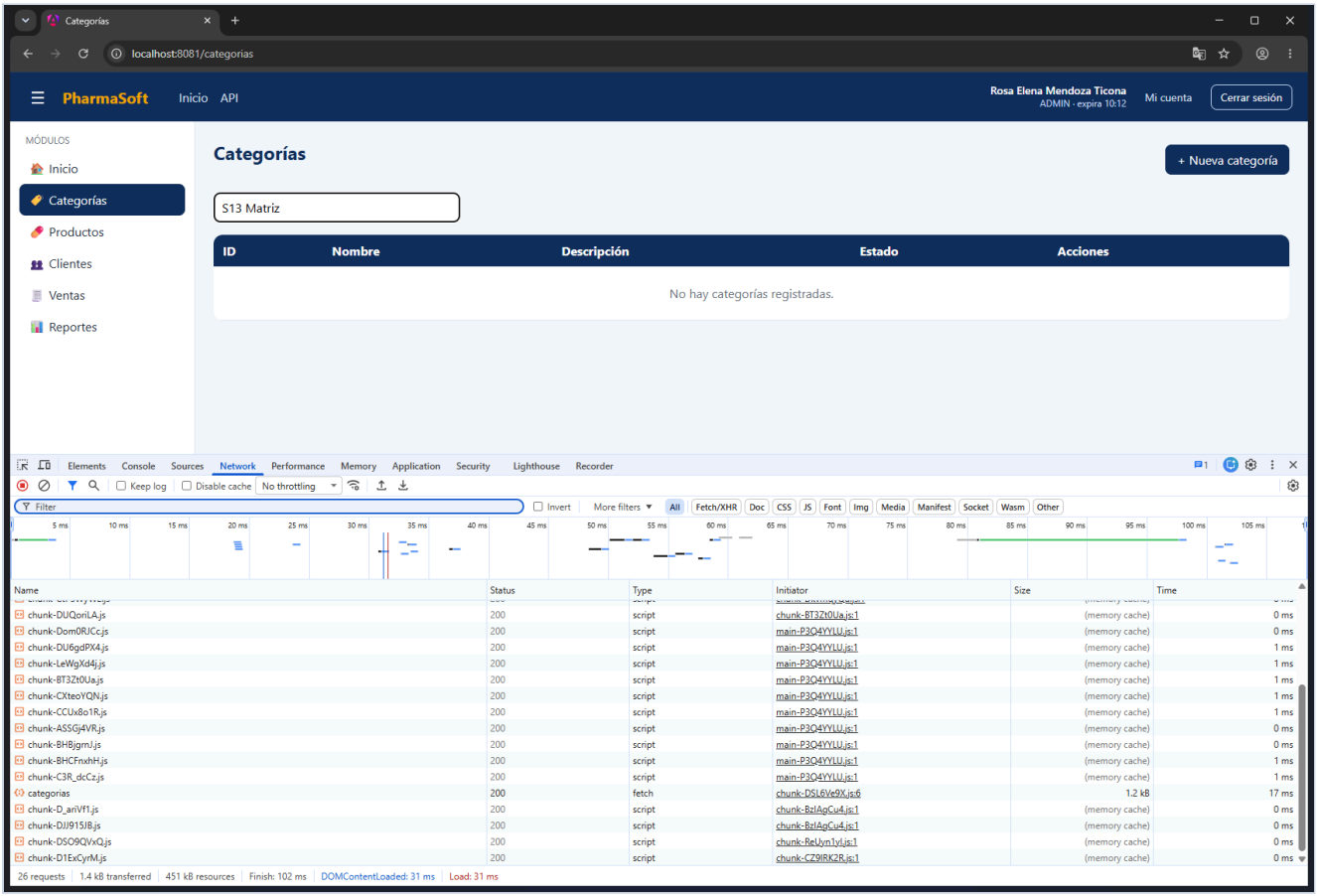
C-03b-precio-api.png · C-03, API.



C-04-stock.png · C-04, stock descontado.



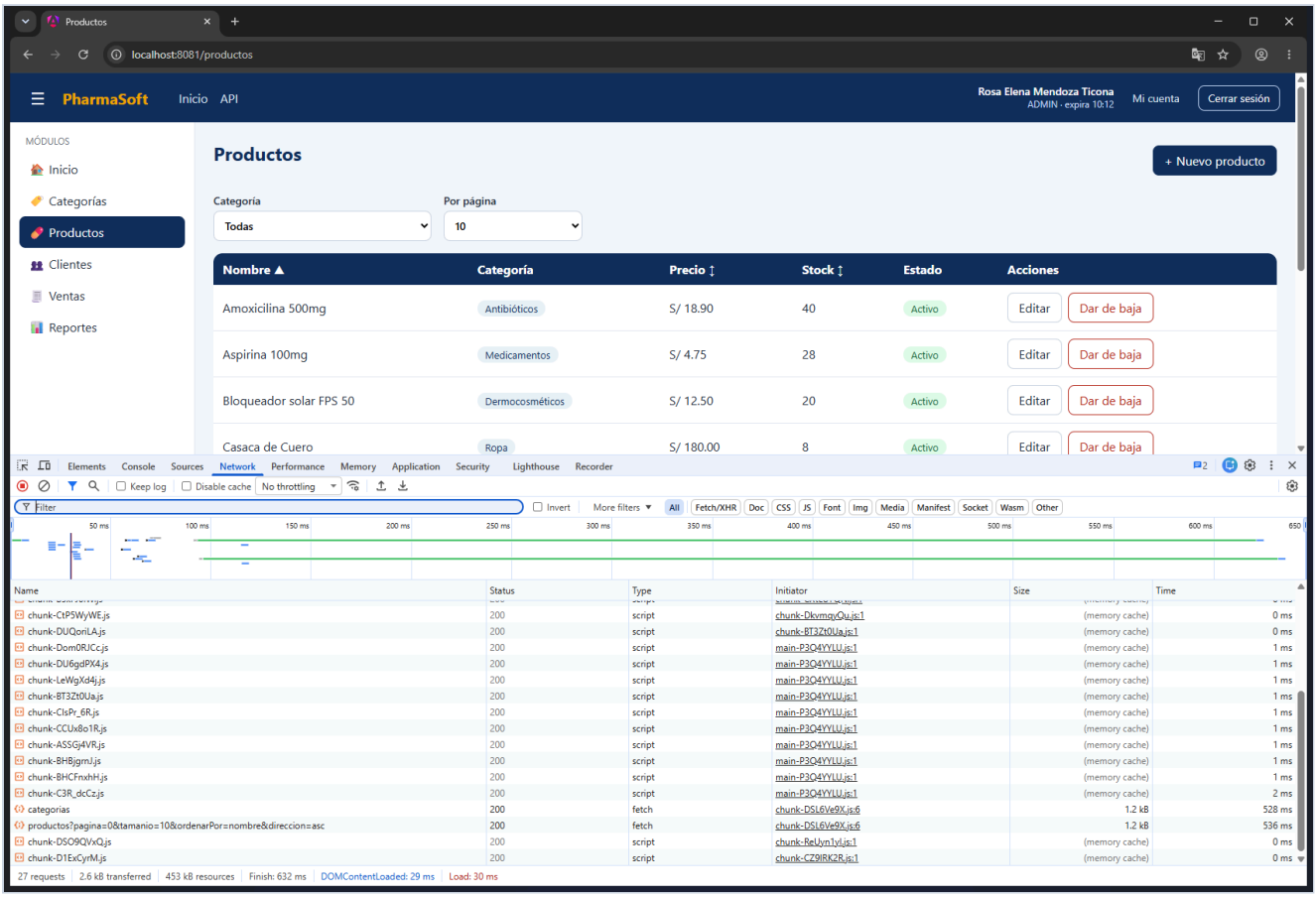
C-04b-stock-tras-anular.png · Extra: stock tras anular.



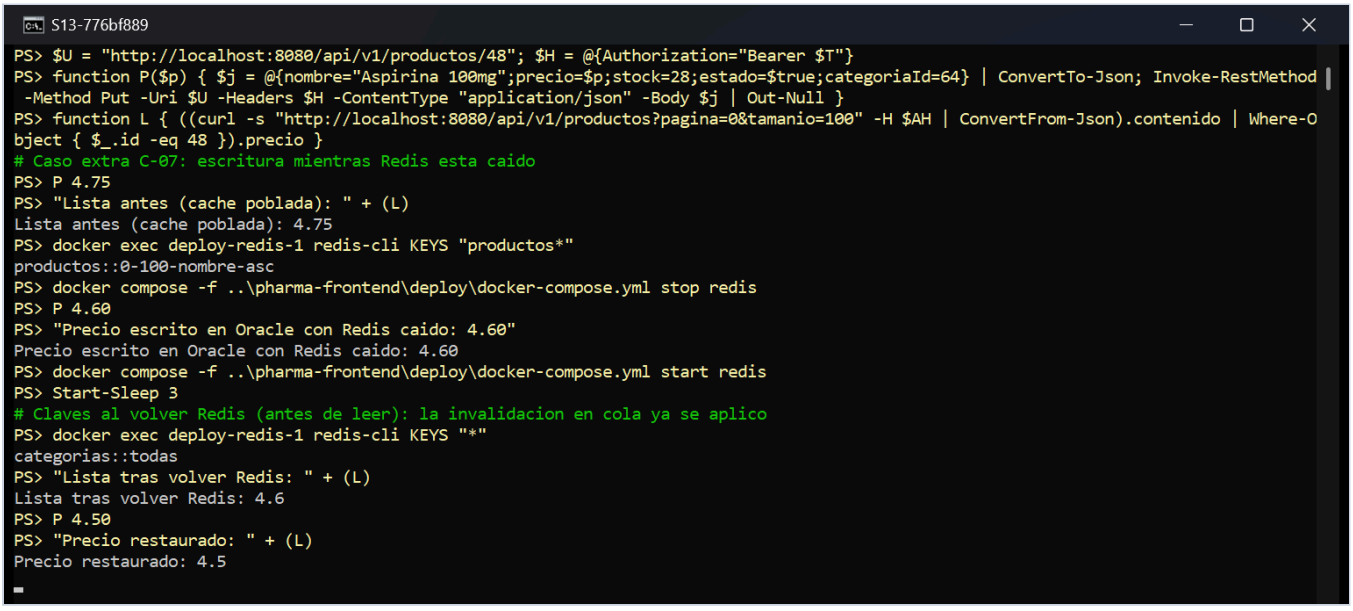
C-05-eliminada.png · C-05.



C-06a-redis-detenido.png · C-06, consola.



C-06b-spa-sin-redis.png · C-06, SPA.



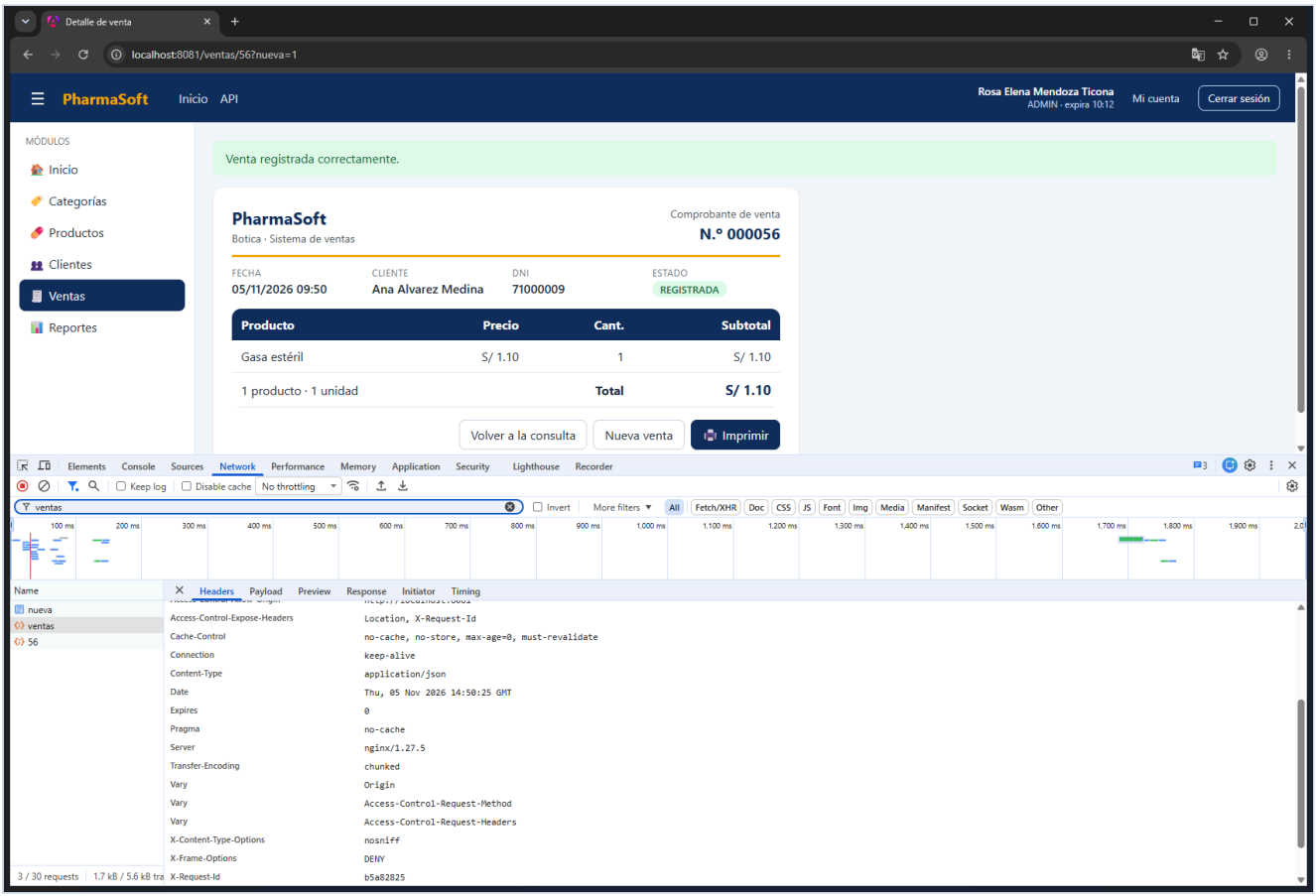
C-07-escritura-con-redis-caido.png · C-07.

Parte D. Logs y monitoreo

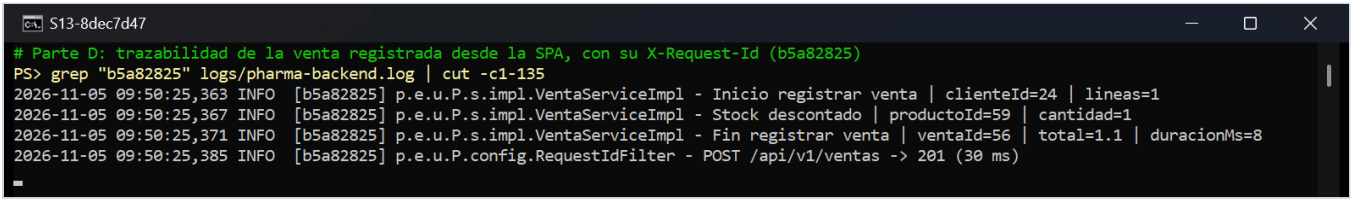
Trazabilidad de una venta

En producción el log solo decía `POST /api/v1/ventas -> 201`, insuficiente para reconstruir una venta. Se añadieron tres líneas a `registrar`. La venta 56, registrada desde la SPA con `X-Request-Id: b5a82825`, queda así:

Línea	Qué cuenta
`Inicio registrar venta | clienteId=24 | lineas=1`	Llegó la petición: quién compra y cuántos productos.
`Stock descontado | productoId=59 | cantidad=1`	Se descontó el stock de ese producto. Hay una línea por producto.
`Fin registrar venta | ventaId=56 | total=1.1 | duracionMs=8`	Número de venta, total y tiempo del servicio.
`POST /api/v1/ventas -> 201 (30 ms)`	Línea de acceso: resultado HTTP y tiempo total, con los filtros de seguridad.



D-01a-x-request-id-venta.png · X-Request-Id en la pestaña Red.



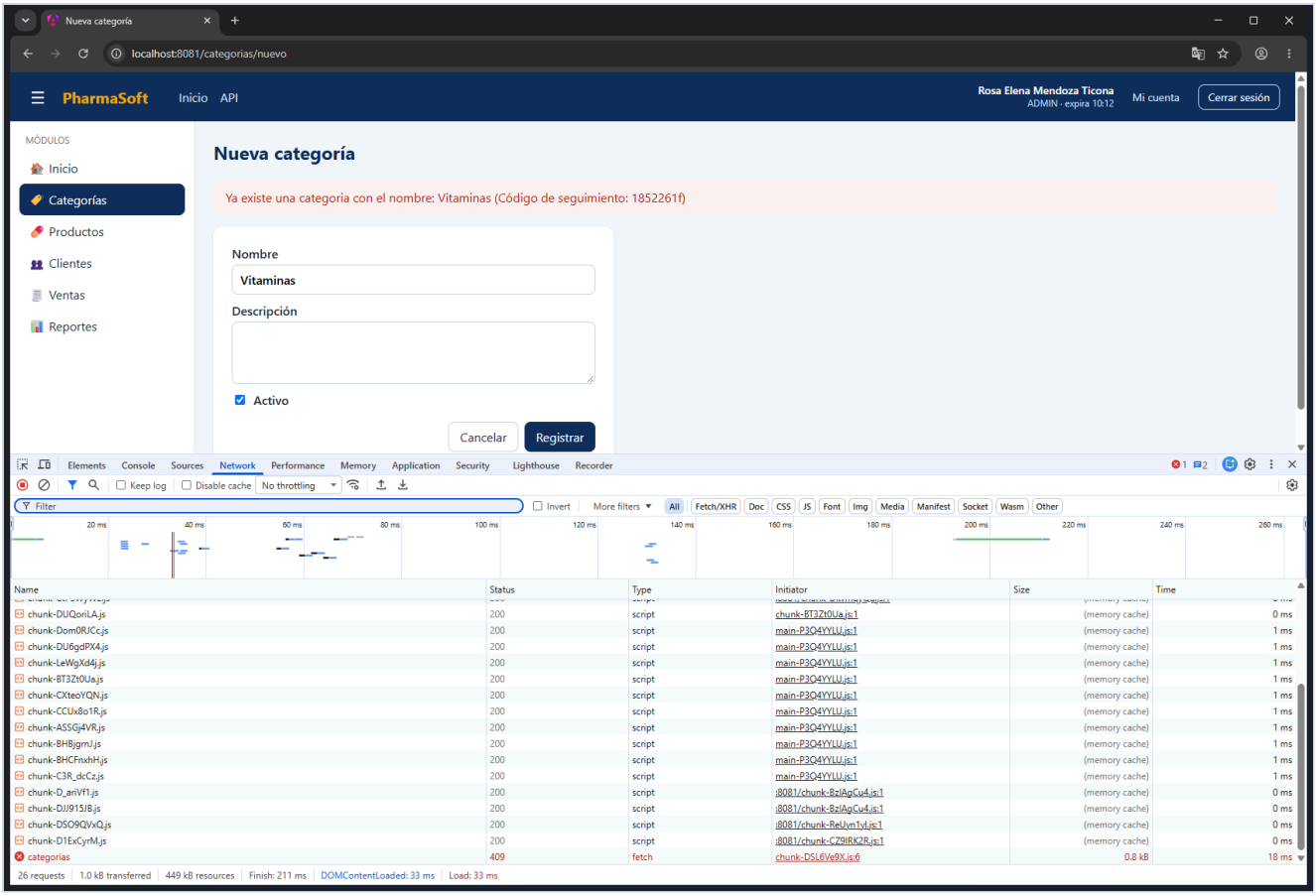
D-01b-venta-en-log.png · grep con ese identificador.

Identificador en los errores

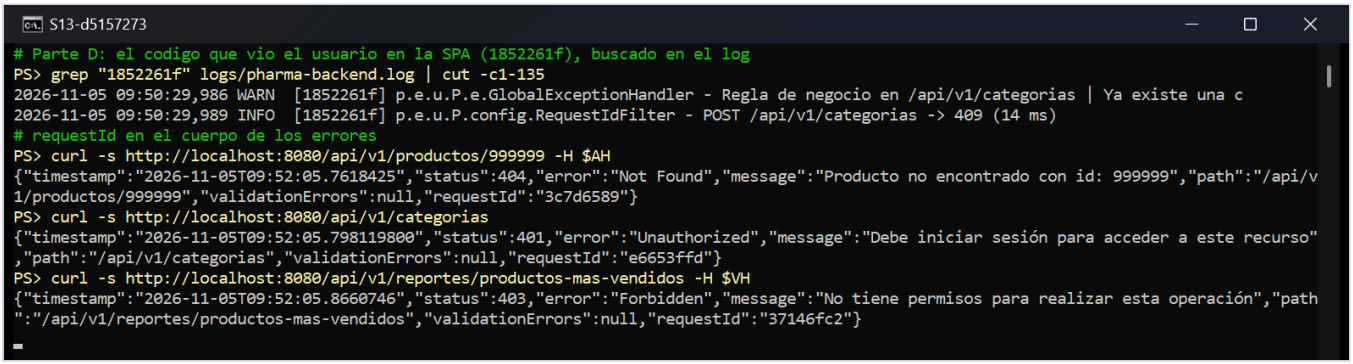
Se agregó `requestId` a `ErrorResponseDTO` y se toma de `MDC.get("requestId")` en `GlobalExceptionHandler` y en `RespuestaErrorSeguridad` (401 y 403 de los filtros). La SPA lo muestra desde `mensajeError`, de modo que los

21 usos existentes lo incluyen sin más cambios. Si el cuerpo no lo trae (500 inesperado, JSON mal formado), lo toma de la cabecera.

Cómo ayuda al soporte: el vendedor dicta «Código de seguimiento: 1852261f» y soporte encuentra con `grep` exactamente las líneas de esa petición, sin buscar por hora entre las de los tres locales.



D-02a-codigo-en-spa.png · Mensaje de error con código de seguimiento.



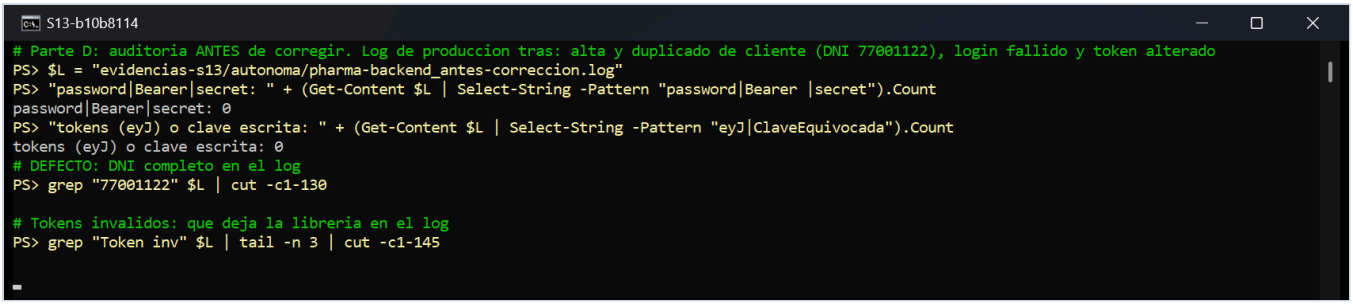
D-02b-codigo-en-log-y-api.png · El código en el log, y requestId en errores 404, 401 y 403.

Auditoría de secretos

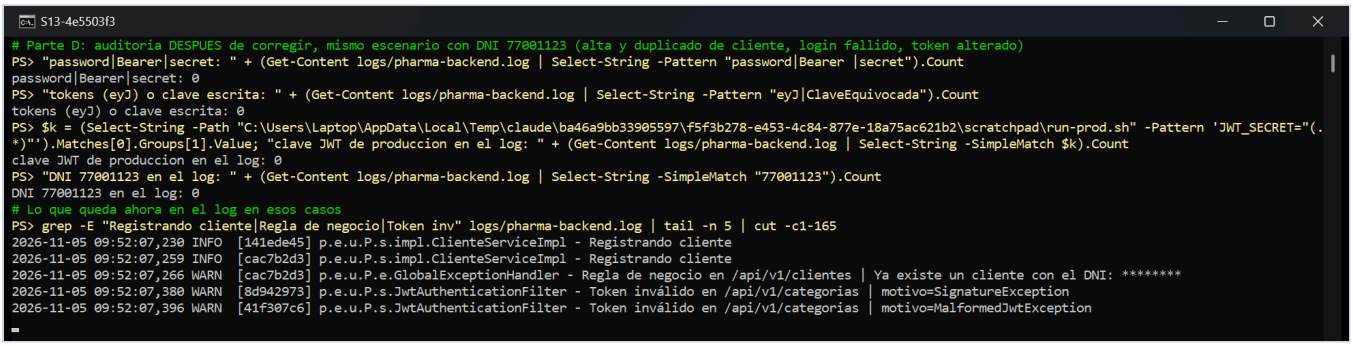
Escenario: alta y duplicado de cliente, login fallido, token con la firma alterada y token mal formado.

Se busca	Antes	Después
`password`, `Bearer ` , `secret`	0	0
Tokens (`eyJ`) o la contraseña escrita	0	0
Clave JWT de producción	No medido	0
DNI completo	**Sí** (2 líneas)	**0**

- **Defecto corregido:** `ClienteServiceImpl` registraba `Registrando cliente con DNI=77001122`. Ahora registra `Registrando cliente`.
- **Prevención:** el nuevo log de reglas de negocio copia un mensaje que puede incluir un DNI («Ya existe un cliente con el DNI…»), así que ocho dígitos seguidos se enmascaran como `********`. El usuario sigue viendo el mensaje completo.
- **Prevención:** un token inválido registraba el mensaje de la librería, que en un token mal formado incluye «Unable to deserialize:». Ahora se registra solo el tipo de error.
- **Observación:** el log conserva el nombre de usuario en cada inicio de sesión. Si alguien escribe su contraseña en el campo de usuario, quedaría registrada; se deja como riesgo conocido.



D-03a_auditoria_antes.png · Antes: aparece el DNI.



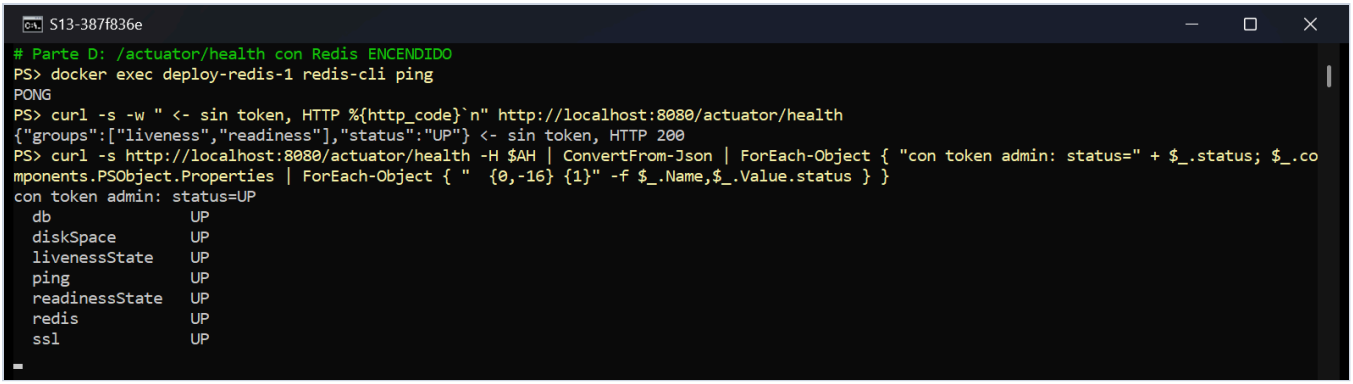
D-03b-auditoria-despues.png · Después: mismo escenario con otro DNI.

Monitoreo

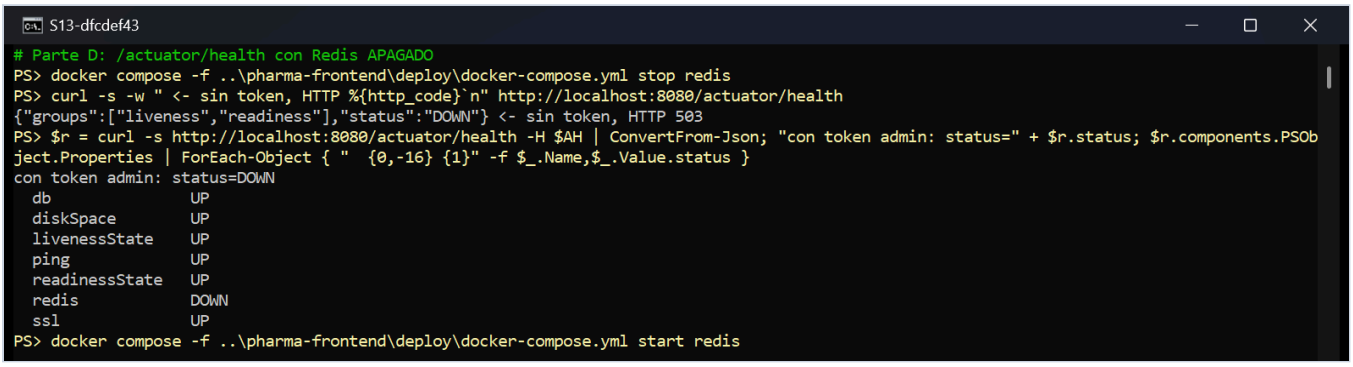
Redis	Sin token	Con token de admin
Encendido	200, `status: UP`, sin componentes	UP; db, redis, diskSpace y demás en UP
Apagado	503, `status: DOWN`, sin componentes	DOWN; **solo redis en DOWN**, db y resto en UP

Interpretación: sin token se ve lo justo para una sonda, sin revelar la infraestructura; con token, se sabe cuál pieza falló. Con Redis apagado la aplicación sigue atendiendo (C-06), pero health informa DOWN y responde 503: un balanceador que use este endpoint retiraría una instancia que funciona. Conviene separar Redis del grupo de `readiness`.

Métrica `http.server.requests?tag=uri:/api/v1/ventas`: 2 peticiones POST con 201, total 0.094 s, media 46.8 ms, máximo 29.4 ms. El máximo es menor que la media porque Micrometer calcula `MAX` sobre una ventana reciente, mientras que `COUNT` y `TOTAL_TIME` acumulan desde el arranque: la primera venta, más lenta, ya salió de esa ventana. Hay pocas peticiones, así que la media no es representativa. La métrica sirve para vigilar tendencias, por ejemplo el aumento de errores 5xx en `outcome`.



D-04a-health-redis-arriba.png · Health con Redis encendido.



D-04b-health-redis-apagado.png · Health con Redis apagado.



D-04c-metricas-ventas.png · Métrica filtrada por uri.

Parte E. Recomendaciones de despliegue

La sección «Despliegue en producción» está en el `README.md` de cada repositorio, con el diagrama, los puertos, las variables sin valores reales, los pasos para desplegar y volver atrás, y el estado de las buenas prácticas. Las tres recomendaciones priorizadas son:

1. **HTTPS en Nginx.** El token y la contraseña viajan hoy sin cifrar.
2. **Un solo backend, Oracle y Redis centrales, con copias de seguridad diarias y prueba de restauración.** Con instalaciones por local, el stock no coincidiría y una venta no invalidaría la caché de los demás.
3. **Alertas sobre `/actuator/health`.** Hoy un fallo se descubre cuando un vendedor lo reporta.

Parte F. Versionamiento y pruebas

- **Ramas:** `feature/optimizacion-produccion-tineo` en los dos repositorios.
- **Solicitudes:** frontend hacia `develop` (https://github.com/Legends600/pharma-frontend/pull/14). Backend hacia `develop` (https://github.com/Legends600/TareaS1/pull/7); la rama se creó desde `main` porque ese repositorio no la tenía.
- **Commits:** al menos tres por repositorio, con la convención de la guía, todos del 5 de noviembre de 2026. El docente indicó que no es necesario hacerlos en días distintos.

- **Pruebas automáticas:** 24 en el backend y 5 en el frontend, todas en verde, incluidas las nuevas de `requestId` y del código de seguimiento.
- **Capturas:** ninguna muestra un token ni una contraseña. Las consolas usan variables y el inicio de sesión se hace fuera de pantalla; en DevTools se muestran solo los encabezados de respuesta. Los DNI que aparecen son de clientes de prueba.
- **Datos de prueba:** quedan en Oracle las ventas anuladas 53 a 56 y los clientes de auditoría dados de baja (DNI 77001122 y 77001123). El stock y los precios se restauraron.

Lista de cotejo

N.º	Criterio	Estado
1	Cinco módulos más pesados con el analizador y su origen	Sí
2	Mejora adicional con medición antes y después	Sí
3	Con y sin precarga, justificado con cifras	Sí
4	Política de caché de cada recurso y despliegue simulado	Sí
5	Por qué el API no se guarda en la caché del navegador	Sí
6	Tasa de aciertos y tiempos de fallo y acierto	Sí
7	TTL y seis casos de la matriz	Sí
8	Defectos de invalidación corregidos o justificación	Sí (no se encontraron)
9	Venta reconstruida en el log	Sí
10	requestId en el backend y en la SPA	Sí
11	Auditoría de secretos, health y métricas	Sí
12	Sección de despliegue completa	Sí
13	Commits, entrega y solicitudes	Sí (el docente indicó que los commits pueden ser del mismo día)